SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



11. C++ Ressourcenverwaltung

Überblick: Ressourcenverwaltung in C++ (i)

- manuelles Ressourcenmanagement (wie in C) of umständlich und fehleranfällig
  - viele Spezialfälle zu berücksichtigen
  - Programmierer*in muss (immer wieder) “*an alles denken*”

Sammlung von Ideen und Techniken (Idiomen) zur Vereinfachung

- Ressourcenmanagement möglichst selten (am besten nur einmal) implementieren und wiederverwenden
  - z.B. im Zuge der Entwicklung eigener Klassen
  - Ressourcen binden an Objekt-Lifetime (Konstruktor → Destruktor)
- sinnvoller Umgang mit “*leeren*” Ressourcen, z.B. `nullptr`
  - wenn möglich komplett vermeiden
  - sonst hervorheben und möglichst mit automatischen Variablen abdecken
- Verantwortung für Ressourcenverwaltung (ownership)
  - klares Bild wer “Besitzer” einer Ressource ist
    - “Besitz” reduzieren
  - eindeutige Besitz-Übertragung durch Schnittstellenbeschreibung
    - z.B. »`const`& `T`« vs. »& `T`« vs. `T`

Überblick: Ressourcenverwaltung in C++ (ii)

typische Idiome & Techniken

- Resource Acquisition is Initialization (RAII)
  - eigentlich besser »Resource Release is Destruction«
- Move Semantik
  - effizientes Übertragen von Ressourcen ohne Kopie aber mit Werte-Semantik
- Zusammenhang von speziellen Member-Funktionen, wie Kopierkonstruktor, Destruktor und Zuweisungsoperator
  - Rule-of-0
    - keine speziellen Member
  - Rule-of-3 (vor C++11)
    - Kopierkonstruktor, Zuweisungsoperator und Destruktor (keine oder alle)
  - Rule-of-5 (seit C++11)
    - Rule-of-3 + Move-Kopierkonstruktor und Move-Zuweisungsoperator

Ziel: Ressourcenverwaltung einfach und sicher machen

Lifetime Extension mit `const &`

- Mit Hilfe von const-Referenzen können Lifetimes verlängert werden
- Wiederholung: Beispiel nicht-virtueller Destruktor (Objekt hinter `Base`-Zeiger `b` wird nicht korrekt zerstört)
- Referenz `b2` hingegen verlängert Lebensdauer des referenzierten `Derived()`-Objekts
  - referenziertes `Derived`-Objekt wird korrekt zerstört
  - kein Zeiger notwendig

```
struct Base {
  Base() { std::print("Base(), "); }
  ~Base() { std::print("~Base(), "); }
};
struct Derived : public Base {
  Derived() { std::print("Derived(), "); }
  ~Derived() { std::print("~Derived(), "); }
};
Derived *d = new Derived(); delete d; PL();
Base *b = new Derived(); delete b; PL();

const Base &b2 = Derived(); // lifetime extension
```

```
Base(), Derived(), ~Derived(), ~Base(),
Base(), Derived(), ~Base(),
Base(), Derived(), ~Derived(), ~Base(),
```

https://herbsutter.com/2008/01/01/gotw-88-a-candidate-for-the-most-important-const/

Ressourcenkapselung und Wertesemantik verhindern Probleme

- häufige Probleme können durch Ressourcenkapselung und Wertesemantik (*call by value*) drastisch reduziert werden

- manuelle Ressourcenverwaltung in C
  - eigentliche *Aufgabe* schwer zu erkennen

```c
#include <string.h>

const char hello[] = "Hello, ";
const char world[] = "World!";

// manual allocation & size calculation
char *hello_world =
  malloc(sizeof(char) *
         (strlen(hello) + strlen(world) + 1));
hello_world[0] = '\0'; // manual init

char *end = strcat(hello_world, hello);
strcat(end, world);
SHOW(hello_world);

free(hello_world); // manual free'ing
```

```
hello_world = "Hello, World!"
```

- durch `std::string` wird Speicher reserviert und wieder freigegeben
- durchgehend Wertesemantik
  - Zeiger und Speicherverwaltung im Hintergrund
  - Aufgabe klar ersichtlich

```c
std::string hello {"Hello, "};
std::string world {"World!"};

// automatic allocation (and re-use)
std::string hello_world = hello + world;

SHOW(hello_world);

// automatic deallocation
```

```
hello_world = "Hello, World!"
```

Resource Acquisition is Initialization (RAII)

- wichtiges C++ Idiom für sauberen Code
- Objekte existieren nur in Kombination mit ihren benötigten Ressourcen
- benötigte Ressourcen werden
  - im Konstruktor initialisiert
    - stehen daher während der Lebenszeit immer zu Verfügung
  - im Destruktor freigegeben
    - werden nach der Lebenszeit automatisch freigegeben
- Beispiel C-File-Handle durch `std::fstream` automatisch verwaltet

```
{
  std::ofstream f { "test.txt" }; // open file
  std::println(f, "Hello!");
  // NOTE: automatically closed (and saved) at end of scope
}
std::ifstream f { "test.txt" }; // open again
std::string text;
f >> text; // read text
SHOW(text);
```

```
text = "Hello!"
```

RAII Beispiel `std::vector`

- RAII ist “*komponierbar*”: Container mit RAII verwaltet Elemente mit RAII
- `std::vector` verwaltet internen Speicher (wahrscheinlich dynamisches Array) automatisch
- Kopieren von `vector` kopiert Elemente auch
- einfache Verwendung, ohne auf Speicherverwaltung achten zu müssen

```cpp
int main() {
  std::vector<int> numbers = {1, 2, 3};
  std::vector<int> copy = numbers;
  SHOW(numbers, copy);
}
```

```
numbers = [1, 2, 3]; copy = [1, 2, 3]
```

```cpp
struct N {
  int n;
  N(int nr) : n(nr) { P("N({}), ", n); }
  ~N() { P("~N({}), ", n); }
  N(const N& nr) : n(nr.n) { P("copy N({}), ", n); }
};

int main() {
  std::vector<N> c {1, 2, 3}; PL();
  std::vector<N> c2 = c; PL();
}
```

```
N(1), N(2), N(3), copy N(1), copy N(2), copy N(3),
↪  ~N(3), ~N(2), ~N(1),
copy N(1), copy N(2), copy N(3),
~N(1), ~N(2), ~N(3), ~N(1), ~N(2), ~N(3),
```

RAII Beispiel `std::jthread`

```cpp
using clk = std::chrono::system_clock;
using tp = std::chrono::time_point<clk>;

void calculate() {
  using namespace std::chrono_literals;
  std::this_thread::sleep_for(1s);
}

int main() {
  tp create, begin = clk::now();
  {
    std::vector<std::jthread> threads;
    for (size_t i = 0; i < 1000; i++) {
      threads.emplace_back(calculate);
    }
    create = clk::now();
    // automatically wait for threads (join)
  }
  tp end = clk::now();
  std::println("create {:%S}s", create-begin);
  std::println("total {:%S}s", end-begin);
}
```

- einfache Nebenläufigkeit mit Join-Threads
  - “*join*” steht hier für “Warten auf Ende”
- Join-Thread enthält Thread
  - Destruktor wartet auf Thread-Ende

```
create 00.083100930s
total 01.084574524s
```

- erstellt und startet 1000 Threads
  - erste Zeitmessung (`create`)
- einzelne Threads rechnen (warten 1s)
- wartet (automatisch) auf Threads
  - automatischer Destruktoraufruf von `std::vector` ruft Destruktor aller Elemente (Threads)
  - zweite Zeitmessung (`end`)

RAII Beispiel `std::jthread` & `std::mutex`

- ohne Sperren
  - gegenseitiges Überschreiben
  - falsche Summe

```
int g_counter {};

void count() {
  using namespace std::chrono_literals;
  for (int i=0; i < 100; i++) {
    int temp = g_counter;
    std::this_thread::sleep_for(1ns);
    g_counter = temp + 1;
  }
}
int main() {
  {
    std::vector<std::jthread> threads;
    for (size_t i = 0; i < 100; i++) {
      threads.emplace_back(count);
    }
  }
  SHOW(g_counter);
}
```

```
g_counter = 2681
```

- automatischer Lebensdauer der Sperre
  - innerhalb eines Schleifendurchlaufs
- sehr einfache und sichere Verwendung
  - Sperre wird garantiert freigegeben
    - sogar im Fehlerfall

```
int g_counter {};
std::mutex g_mutex {};

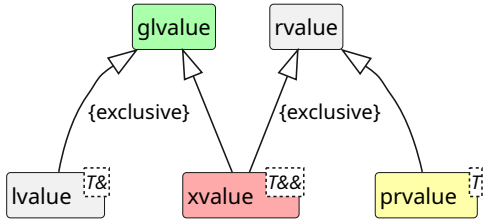
void count() {
  using namespace std::chrono_literals;
  for (int i=0; i < 100; i++) {
    std::lock_guard guard(g_mutex);
    int temp = g_counter;
    std::this_thread::sleep_for(1ns);
    g_counter = temp + 1;
    // automatic release of mutex lock
  }
}
```

C++ Werte-Kategorien (eigentlich Ausdruckskategorien)

- *generalized left value* (glvalue)
  - Ausdrücke deren Evaluierung die Identität eines Objekts bestimmen
  - z.B. linke Seite einer Zuweisung
- *pure right value* (prvalue) Ausdrücke mit Resultat ohne Identität
  - z.B. temporäres (Zwischen-)Ergebnis, rechte Seite einer Zuweisung
  - z.B. Ausdruck in `return`-Anweisung
- *exipiring value* (xvalue) Ausdruck dessen Objekt wiederverwendet werden darf (“*eXpiring*”)

```
int x = 3;
//  ^   ^-- rvalue (right-hand operand)
//  '------ lvalue (left-hand operand)

std::vector<int> v { 1, 2, 3 };
return v;
//     ^-- xvalue
```



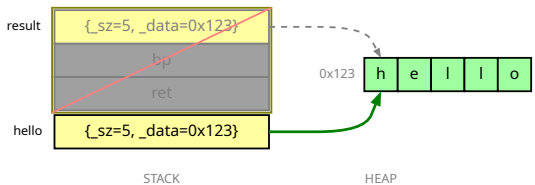
Value categories - cppreference.com

Move-Semantik Idee

- Daten selbst müssen oft nicht kopiert werden → nur (leichtgewichtige) Hülle
- Eigentum (*ownership*) wird übertragen
  - Inhalt wird bewegt → *move*'d
    - eigentliche Daten selbst werden nicht bewegt sondern nur deren Besitz
- z.B. Zeichenkette liegt am Heap, `std::string` ist nur Hülle (Zeiger + Länge)
  - eigentliche Daten müssen nicht kopiert werden

```cpp
std::string make_string() {
  std::string result { "hello" };
  return result; // NOTE: automatic move
}

int main() {
  std::string hello = make_string();
}
```



Move-Semantik

- explizit erlaubtes Bewegen von Ressourcen, z.B. dyn. Speicher am Heap
- Ressource wird von einem Besitzer zum nächsten übergeben (bewegt)
- beide Teilnehmer bleiben *informiert*
  - z.B. Elemente werden von einer Collection zur anderen transferiert
  - müssen dabei nicht kopiert werden, vorheriger Besitzer ist danach *“leer”*
- Kennzeichnung in Funktionsschnittstelle durch »`&&`«
- wichtigste Funktionen
  - `std::move<T>(T &&x)` markiert `x` als bewegbar
  - `std::swap<T>(T &x, T &y)` vertauscht Inhalte von `x` und `y`

```
std::vector<int> v1 {1, 2, 3}, v2;  SHOW(v1, v2);
v2 = std::move(v1); /* v1 --> v2 */ SHOW(v1, v2);
v1.push_back(0);                    SHOW(v1, v2);
std::swap(v1, v2); /* v1 <--> v2 */ SHOW(v1, v2);
```

```
v1 = [1, 2, 3]; v2 = []
v1 = []; v2 = [1, 2, 3]
v1 = [0]; v2 = [1, 2, 3]
v1 = [1, 2, 3]; v2 = [0]
```

- Implementierung für eigene Klassen mit
  - Move-Konstruktor, z.B. `Complex(Complex &&c)`
  - Move-Zuweisungsoperator `Complex& operator = (Complex &&c)`
- ermöglicht automatische effiziente Wiederverwendung von Ressourcen

Ohne Move Operationen

- ohne Move-Semantik, unnötiges Kopieren, obwohl nur eine Instanz benötigt wird
  - außer `in Zeile 22: return &&temp` und `Zeile 27: Value v2(&&)`

```
1  struct Value {
2    std::string m_value {};
3    Value(std::string value) : m_value{value} {
4      PL("Value({})", value);
5    }
6    Value(const Value &v) : m_value{v.m_value} {
7      PL("Value({}) copied", m_value);
8    }
9    ~Value() {
10     PL("~Value({}) destroyed", m_value);
11   }
12   Value& operator = (const Value& v) {
13     PL("{}<-{} copy assigned", m_value, v.m_value);
14     m_value = v.m_value;
15     return *this;
16   }
17 };
18 Value pass_value(Value v) { // copy 1b
19   NOTE("inside call, before copy ctor");
20   Value temp = v; // copy 2
21   NOTE("inside call, before return (v dtor)");
22   return temp;
23 }
```

```
24 int main() {
25   Value v {"test"};
26   NOTE("before call");
27   Value v2 =
28     pass_value(v);  // copy 1a
29   NOTE("after call");
30   v = v2; // copy 3
31   NOTE("end");
32 }
```

```
Value(test)
// before call
Value(test) copied
// inside call, before copy ctor
Value(test) copied
// inside call, before return (v dtor)
~Value(test) destroyed
// after call
test<-test copy assigned
// end
~Value(test) destroyed
~Value(test) destroyed
```

Mit Move Operationen

- mit Move-Semantik, (teilweise) automatisches Wiederverwenden
  - keine Kopien → “*Hüllen*” werden “leer geräumt”

```
1  struct Value {
2    // ...
3    // move ctor
4    Value(Value &&v): m_value{std::move(v.m_value)} {
5      PL("Value({}) move ctor", m_value);
6    }
7    // move assignment
8    Value& operator = (Value&& v) {
9      PL("{}<-{} move assigned", m_value, v.m_value);
10     m_value = std::move(v.m_value);
11     return *this;
12   }
13 };
```

```
14 Value pass_value(Value &&v) {
15   NOTE("inside call, before move ctor");
16   Value temp = std::move(v);
17   NOTE("inside call, before return");
18   return temp; // NOTE: move
19 }
```

```
20 int main() {
21   Value v {"test"};
22   NOTE("before call");
23   Value v2 =
24     pass_value(std::move(v));
25   NOTE("after call");
26   v = std::move(v2);
27   NOTE("end");
28 }
```

```
Value(test)
// before call
// inside call, before move ctor
Value(test) move ctor
// inside call, before return
// after call
<-test move assigned
// end
~Value() destroyed
~Value(test) destroyed
```

Beispiel Move Operationen selbst implementieren (i)

```
int* dupe(int* p);

class R {
 int* m_ptr {};
 std::string s() const;

public:
 R() = default;

 R(int i) : m_ptr(new int{i}) {
  P("R({}), ", s());
 }

 ~R() {
  P("~R({}),", s());
  delete m_ptr;
 }

 R(const R &r)
  : m_ptr(dupe(r.m_ptr)) {
  PL("R({}) copy ctor", s());
 }
```

```
 R& operator = (const R &r) {
  PL("{}<-{} copy", s(), r.s());
  if (this != &r) {
   delete m_ptr;
   m_ptr = dupe(r.m_ptr);
  } // else -> self assignment
  return *this;
 }

 // move operations
 R(R &&r) : m_ptr(r.m_ptr) {
  PL("R({}) move ctor", s());
  r.m_ptr = nullptr;
 }

 R& operator = (R &&r) {
  PL("{}<-{} move", s(), r.s());
  if (this != &r) {
   delete m_ptr;
   m_ptr = r.m_ptr;
   r.m_ptr = nullptr;
  } // else -> self move
  return *this;
 }
};
```

```
int main() {
 R r1 {1}, r2 {2}; PL("\n");
 PL("---- copy ops ----");
 R r_cp {r1};
 r_cp = r2; PL();
 PL("---- move ops ----");
 R r_mv { std::move(r2) };
 r_mv = std::move(r1); PL();
 PL("---- destructors -----");
 PL("r_mv, r_cp, r2, r1");
}
```

```
R(1), R(2),

---- copy ops ----
R(1) copy ctor
1<-2 copy

---- move ops ----
R(2) move ctor
2<-1 move

---- destructors -----
r_mv, r_cp, r2, r1
~R(1),~R(2),~R(null),~R(null),
```

Beispiel Move Operationen selbst implementieren (ii)

- Hilfsfunktionen für `int`-Zeiger (Duplizieren & Ausgabe)

```
int* dupe(int* p) { return p ? new int{*p} : p; }
std::string R::s() const { return m_ptr ? std::format("{}", *m_ptr) : "null"; }
```

- Selbstzuweisung berücksichtigen
  - ansonsten zuerst löschen des eigenen Inhalts
    - dann Zuweisung ⇒ Ressource verloren

```
R& operator = (const R &r) {
  if (this != &r) { // no self assignment
    delete m_ptr;
    m_ptr = dupe(r.m_ptr);
  }
  return *this;
}
```

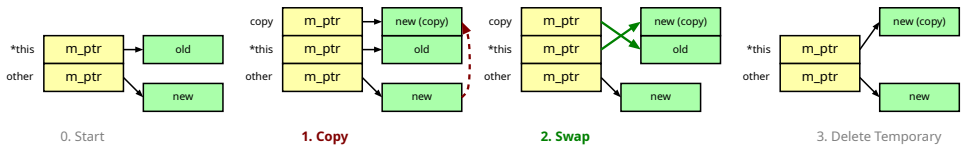
- Selbst-Move ebenfalls ausschließen
  - ansonsten zuerst löschen des eigenen Inhalts
    - dann Bewegen der gelöschten Ressource

```
R& operator = (R &&r) {
  if (this != &r) { // no self assignment
    delete m_ptr;
    m_ptr = r.m_ptr;
    r.m_ptr = nullptr;
  }
  return *this;
}
```

- umständlich und daher fehleranfällig → Copy & Swap Idiom

Copy-and-Swap Idiom

- Implementierung von Zuweisungsoperationen umständlich und fehleranfällig
- Idee: Kopierkonstruktor und `swap()`-Funktion zur Vereinfachung
  - 1. Kopieren/Bewegen in ein temporäres Objekt am Stack
  - 2. Austauschen der Inhalte mit `swap()`
  - 3. automatische & korrekte Freigabe der Kopie durch RAII



Exkurs: `swap()`-Funktionen

- jeder mögliche Typ benötigt eigene `swap()` Funktion um Inhalte zu vertauschen
  - viele Implementierungen bereits in Standardbibliothek

```
auto a = "a";
auto b = "b";
std::swap(a, b);
SHOW(a, b);
```

```
a = "b"; b = "a"
```

- Problem: sehr viele ähnliche `swap()`-Funktionen, leichte Verwechslung möglich
- Idee: Mechanismus ähnlich zu Methodenaufruf zum Finden der passenden Funktion
  - Exkurs: Argument-Dependent Lookup (ADL)

Exkurs: Argument-Dependent Lookup (ADL)

- nicht sichtbare Funktionen können aufgrund des Typs der Parameter gefunden werden

```cpp
namespace SWC3 {
  class Complex {};
  void print() {
    std::println("print()");
  }
  void print(const Complex &c) { // <-----.
    std::println("print(Complex)"); //     |
  }                                //     |
}                                  //     |
                                   //     |
int main() {                       //     |
  // regular qualified calls               |
  SWC3::print();                   //     |
  SWC3::print(SWC3::Complex{});    //     |
  // ADL call                              |
  print(SWC3::Complex{}); // -------------'
  // print(); // XXX: print was not declared
}
```

```
print()
print(Complex)
print(Complex)
```

- vergleichbar mit Methodenaufruf

```cpp
struct Complex {
  void print() const {
    std::println("Complex::print() method");
  }
  // NOTE: function only callable via ADL
  friend void print(const Complex &c) { // <----.
    std::println("print(Complex) function"); // |
  }                                           // |
  friend void print() { } //  <------.         |
};                        //         |         |
                          //         |         |
int main() {              //         |         |
  Complex c {};           //         |         |
  c.print();              //         |         |
  print(c); // ----------------------+---------'
  // print(); // XXX: not found -----'
}
```

```
Complex::print() method
print(Complex) function
```

Argument-dependent lookup - cppreference.com

Copy-and-Swap Idiom für Zuweisungsoperatoren

```
int* dupe(int* p);

class R {
 int* m_ptr {};
 std::string s() const;
public:
 R() = default;

 R(int i) : m_ptr(new int{i}) {
   P("R({}) ", s());
 }

 ~R() { P("~R({}) ", s());
  delete m_ptr;
 }

 R(const R &r)
  : m_ptr(dupe(r.m_ptr)) {
  PL("R({}) copy ctor", s());
 }

 R(R &&r) : m_ptr(r.m_ptr) {
  r.m_ptr = nullptr;
  PL("R({}) move ctor", s());
 }
```

```
 R& operator = (const R &r) {
  PL("copy {}<-{}:", s(), r.s());
  R copy {r};
  swap(*this, copy);
  return *this;
 }

 R& operator = (R &&r) {
  PL("move {}<-{}:", s(), r.s());
  R copy { std::move(r) };
  swap(*this, copy);
  return *this;
 }

 friend // function
 void swap(R &a, R &b) {
    PL("swap {}<->{}",a.s(),b.s());
    using std::swap;
    swap(a.m_ptr, b.m_ptr);
 }

};
```

```
int main() {
 R r1{1}, r2{2}, r3{3}; PL();
 PL("---- copy assign ----");
 r2 = r1; PL();
 PL("---- move assign ----");
 r3 = std::move(r1); PL();
 PL("---- destructors -----");
 PL("r3, r2, r1");
}
```

```
R(1) R(2) R(3)
---- copy assign ----
copy 2<-1:
R(1) copy ctor
swap 2<->1
~R(2)
---- move assign ----
move 3<-1:
R(1) move ctor
swap 3<->1
~R(3)
---- destructors -----
r3, r2, r1
~R(1) ~R(1) ~R(null)
```

Rule-of-0

- Wenn möglich
  - Wertesemantik für all Klassen-Member
  - Initialisierung bei Member-Definition
- keine speziellen Methoden notwendig (Kopierkonstruktor, Destruktor, Zuweisungsoperator, ...)
- einfache Datenkapsel
- Kopie von Klasse erzeugt Kopie von Komponente
- empfehlenswert für einfache (und kleinere) Klassen

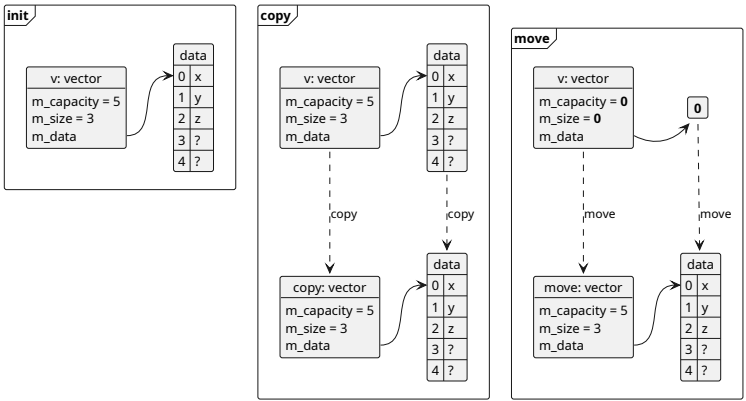
```
class Complex {
  double m_re {}, m_im {};
public:
  friend std::ostream& operator << (std::ostream &os, const Complex &c) {
    return os << '(' << c.m_re << ',' << c.m_im << ')';
  }
};
int main() {
  Complex c {}; // default ctor
  Complex c2 {c}; // default copy ctor
  c = c2; // default assignemnt operator
  SHOW(c, c2);
}
```

Rule-of-5 (vor C++11 Rule-of-3)

- Falls eine der folgenden speziellen Member implementiert wird, sollten sehr wahrscheinlich auch alle anderen implementiert werden
  - Kopierkonstruktor
  - Zuweisungsoperator
  - Destruktor
  - Move-Konstruktor (seit C++11)
  - Move-Zuweisungsoperator (set C++11)
- Hintergrund: sobald manuelle Ressourcenverwaltung notwendig ist müssen diese Funktionen überarbeitet werden
  - z.B. statt Adressen von Zeigern oder Feldern dahinter liegende Daten kopieren (shallow $\rightarrow$ deep copy)

Beispiel: Rule-of-5

- explizite Ressourcenverwaltung notwendig, z.B. Zeiger oder Array
  - Kopieren erfordert Deep-Copy
  - Move erfordert “Leeren” der Quelle



Smart-Pointer Idee

- C++ erlaubt Überladen von Operatoren
- ermöglicht das Verhalten von Zeiger zu imitieren
- z.B. Verwendung von Zeiger-Operationen ohne Zeiger oder dynamischen Speicher
- Kapselung des dynamischen Speichers mit RAII in Smart-Pointer Objekt

```
class Pointer {
  int m_value;
public:
  Pointer(int value) : m_value(value) {}
  int& operator * () {
    return m_value;
  }
};
int main() {
  Pointer p(5);
  SHOW(*p);
  *p = 10;
  SHOW(*p);
}
```

```
*p = 5
*p = 10
```

Überblick Smart-Pointer

- Zeiger mit eindeutigem Besitz (`std::unique_ptr`)
  - pro echtem Zeiger gibt es einen Smart-Pointer
  - kann nicht kopiert werden muss von Besitzer zu Besitzer bewegt werden (move)
  - einfach und effizient, gute Standardlösung
  - gemeinsame Nutzung (sharing) nicht möglich
- Zeiger mit gemeinsamer Nutzung (`std::shared_ptr`)
  - pro echtem Zeiger kann es mehrere Smart-Pointer Objekte geben
  - gemeinsame Verwaltung, z.B. zählen der Referenzen
  - kann beliebig kopiert werden (kann auch bewegt werden)
  - höherer Verwaltungsaufwand aber flexible Nutzung
- Zeiger ohne Besitz (`std::weak_ptr`)
  - gemeinsame Nutzung wie shared pointer
  - keinen Einfluss auf Lebensdauer des dahinter liegenden Zeigers
  - keine Garantie dass dahinter liegender Zeiger noch gültig ist

Nackte Zeiger lassen sich daher fast immer durch Smart-Pointer ersetzen

- Standardlösung: `std::unique_ptr`
- falls gemeinsame Nutzung erforderlich: `std::shared_ptr`
  - um Zyklen zu durchbrechen `std::weak_ptr`

Unique Pointer

- Polymorphismus braucht Referenz (dynamische Bindung) → Smart Pointer als Wrapper
- `std::make_unique` statt Unique-Pointer-Konstruktor erhöht Komfort und Sicherheit

```cpp
struct Shape {
  virtual ~Shape() {}
  virtual void draw() const = 0;
};
struct Rect : public Shape {
    Rect() { P("Rect(), "); }
    ~Rect() { P("~Rect(), "); }
    void draw() const override { P("Rect::draw(),
    ↪   "); }
};
```

```cpp
int main() {
 // std::unique_ptr<Shape> s { new Rect() };
 std::unique_ptr<Shape> s =
 ↪   std::make_unique<Rect>();
 s->draw();
 // std::unique_ptr<Shape> s2 {s}; // XXX: no copy
 std::unique_ptr<Shape> s2 {std::move(s)};
}
```

```
Rect(), Rect::draw(), ~Rect(),
```

```cpp
int main() {
  std::vector<std::unique_ptr<Shape>> shapes;
  shapes.push_back(std::make_unique<Rect>());
  shapes.push_back(std::make_unique<Rect>());
  for (const auto& shape : shapes) {
    shape->draw();
  }
  // copy not allowed
  // std::unique_ptr<Shape> s = shapes[0]; // XXX
}
```

```
Rect(), Rect(), Rect::draw(), Rect::draw(), ~Rect(),
↪   ~Rect(),
```

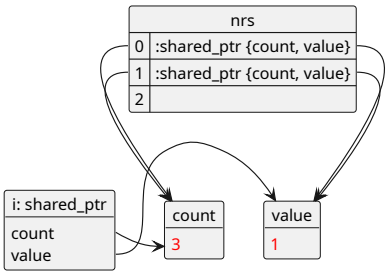
- Unique-Pointer muss bewegt werden (move) und kann nicht kopiert werden
- Verwendung daher als `Shape&`

Beispiel Shared Pointer

- Beliebiges “kopieren” möglich: kopiert nicht den Wert sondern erhöht nur einen Referenzzähler

```
int main() {
  std::shared_ptr<int> i = std::make_shared<int>(0);
  std::vector<std::shared_ptr<int>> nrs { i, i };
  SHOW(*nrs[0], *nrs[1]);
  *i = 1;
  SHOW(*nrs[0], *nrs[1]);
}
```

```
*nrs[0] = 0; *nrs[1] = 0
*nrs[0] = 1; *nrs[1] = 1
```



- letzte Referenz ruft automatisch `delete` auf

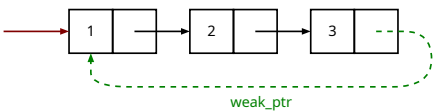
Beispiel Weak Pointer

- Verhalten ähnlich `std::shared_ptr` aber ohne Einfluss auf Referenz-Zähler

```
std::weak_ptr<int> w;
{
  std::shared_ptr<int> s = std::make_shared<int>(1);
  w = s; // downgrade to weak pointer
  SHOW(*s);
  // try upgrade to shared_ptr
  if (std::shared_ptr<int> s2 = w.lock()) {
    SHOW(*s2, w.expired());
  }
  // weak pointer does not extend lifetime of int in s
}
if (std::shared_ptr<int> s2 = w.lock()) {
  SHOW(*s2, w.expired());
} else {
  PL("pointer expired: {}", w.expired());
}
```

```
*s = 1
*s2 = 1; w.expired() = false
pointer expired: true
```

- z.B. Zyklen von `std::shared_ptr` mit `std::weak_ptr` unterbrechen



Smart-Pointer eigene Löschfunktion (*custom deleter*)

- eigene Funktion zum Freigeben einer Ressource bei Verwendung von Smart-Pointer

```
void close(FILE* f) { fclose(f); PL("file closed"); }

int main() {
  { //
    std::unique_ptr<FILE, void(*)(FILE*)> f {
    //                    ^~~~~~~~~~~~~~ type of deleter function
      fopen("test.txt", "w"), //< create FILE pointer
      close //< set custom deleter function close
    };
    fprintf(f.get(), "hello?");
    // NOTE: file closed automatically
  }
  std::ifstream f {"test.txt"};
  std::string s;
  f >> s;
  SHOW(s);
}
```

```
file closed
s = "hello?"
```

Zusammenfassung

- C++ bietet viele Möglichkeiten zur einfachen, sicheren und performanten Ressourcenverwaltung
- Ressourcen an Objektlebenszeit binden mit RAII
- bei eigenen Klassen Rule-of-0 anwenden
  - ansonsten Rule-of-5 beachten
- mit Hilfe von Move-Semantik auch bei umfangreichen Ressourcen effiziente Wertesemantik möglich
- nackte Zeiger (*raw pointer*) dank Smart-Pointer meist vermeidbar

weiteres Material unter C++ Core Guidelines: Resource Management